

# Introdução

Seja bem-vindo ao **Guia Rápido de Java!**

Este livro foi produzido para te apresentar conceitos muito importantes para quem está iniciando na linguagem.

Claro que coloquei uns mais avançados no fim para que você possa se desafiar, e já enxergar os próximos passos, que é o que você quer, certo? =)

Antes de seguir com a leitura do seu material, recomendo que me siga nas redes sociais:

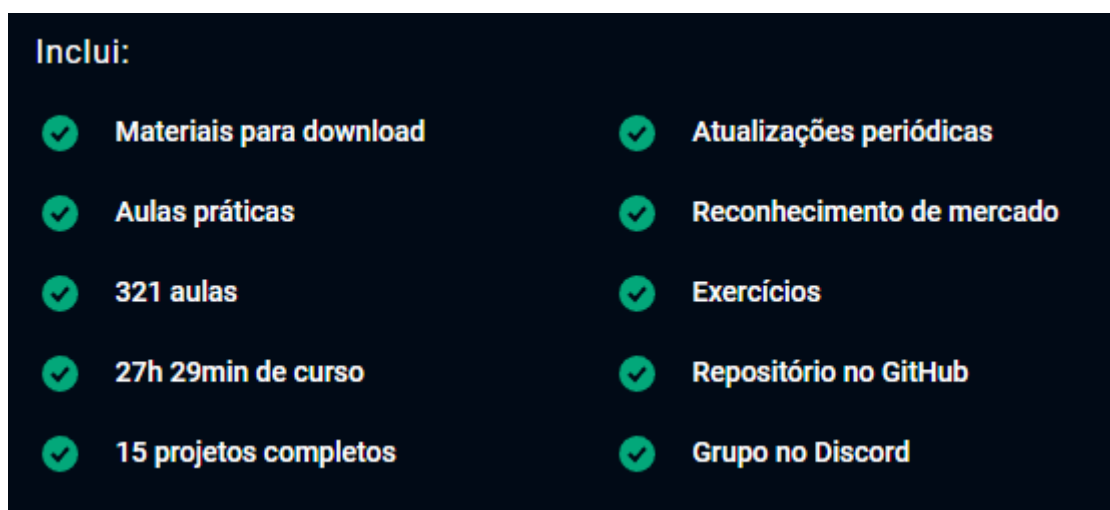
[Canal do YouTube:](#) para mais cursos gratuitos

[Instagram:](#) para falar comigo diretamente, através de lives e caixinhas de pergunta

E se você quiser aprender Java do básico ao avançado, deixo aqui a indicação do nosso curso completo:

## [\*Curso de Java do Básico ao Avançado\*](#)

O curso tem quase 30 horas de duração, diversos exercícios e projetos para você dominar a linguagem.



**Inclui:**

|                           |                             |
|---------------------------|-----------------------------|
| ✓ Materiais para download | ✓ Atualizações periódicas   |
| ✓ Aulas práticas          | ✓ Reconhecimento de mercado |
| ✓ 321 aulas               | ✓ Exercícios                |
| ✓ 27h 29min de curso      | ✓ Repositório no GitHub     |
| ✓ 15 projetos completos   | ✓ Grupo no Discord          |

Agora sim podemos seguir, abra seu VS Code e vamos lá!

### **Sobre este material**

A linguagem de programação Java é uma das mais influentes e amplamente utilizadas no cenário tecnológico atual.

Desde sua criação em meados da década de 1990, Java tem sido a espinha dorsal de inúmeras aplicações, desde sistemas corporativos robustos até aplicativos móveis e soluções web interativas.

Sua filosofia "escreva uma vez, execute em qualquer lugar" (tradução livre de write once, run everywhere) revolucionou o desenvolvimento de software, tornando-se uma escolha preferencial para desenvolvedores que buscam portabilidade e desempenho.

Este **Guia Rápido de Java** foi criado para oferecer a você uma compreensão sólida e prática dos conceitos essenciais que todo programador Java deve dominar.

Os 8 Fundamentos foram divididos em 8 capítulos:

1. **Sintaxe Básica e Estrutura de um Programa Java:** Compreenda a arquitetura fundamental de programas Java e as convenções que tornam o código legível e sustentável;
2. **Tipos de Dados e Variáveis:** Aprenda sobre os tipos primitivos, manipulação de variáveis e conversões de tipos, fundamentais para qualquer aplicação;
3. **Operadores e Expressões:** Descubra como realizar operações matemáticas e lógicas que são o núcleo da tomada de decisões em programas;
4. **Estruturas de Controle de Fluxo:** Explore como direcionar o fluxo de execução através de condicionais e loops, permitindo criar programas dinâmicos e responsivos;
5. **Arrays e Coleções:** Entenda como armazenar e manipular conjuntos de dados, essenciais para lidar com informações em escala;
6. **Fundamentos da Programação Orientada a Objetos:** Mergulhe nos princípios que tornam Java uma linguagem orientada a objetos, facilitando a modularidade e reutilização de código;
7. **Tratamento de Exceções:** Aprenda a construir programas robustos capazes de lidar com erros e situações inesperadas de forma controlada;
8. **Entrada e Saída (I/O):** Conheça as técnicas para interagir com o usuário e manipular arquivos, expandindo as possibilidades de suas aplicações;

Vamos então começar com o primeiro Fundamento!

*Obs:* sugiro ver um por dia, e praticar bastante em cima dele. Assim você fixa o conhecimento e não se sobrecarrega de muito conteúdo.

## Capítulo 1: Sintaxe básica e estrutura de um programa Java

### Estrutura de um programa Java

A linguagem Java é orientada a objetos e possui uma estrutura específica que deve ser seguida. Todo código em Java é escrito dentro de uma classe, e a execução do programa

começa pelo método especial main. Compreender essa estrutura é fundamental para iniciar o desenvolvimento nessa linguagem.

## Classes e métodos

Em Java, uma classe é um modelo que define as características (atributos) e comportamentos (métodos) de um objeto. Aqui está um exemplo simples de uma classe:

```
public class MinhaClasse {
    // Variáveis de instância (atributos)
    int numero;
    String texto;

    // Método construtor
    public MinhaClasse(int numero, String texto) {
        this.numero = numero;
        this.texto = texto;
    }

    // Método (comportamento)
    public void exibirInformacoes() {
        System.out.println("Número: " + numero);
        System.out.println("Texto: " + texto);
    }
}
```

Neste exemplo:

- MinhaClasse é o nome da classe.
- numero e texto são atributos da classe.
- O **construtor** MinhaClasse(int numero, String texto) inicializa os atributos.
- O método exibirInformacoes() exibe os valores dos atributos.

## O método main

O método main é o ponto de entrada de qualquer programa Java. É nele que a execução começa. Sua assinatura padrão é:

```
public static void main(String[] args) {
    // Instruções a serem executadas
}
```

Explicando cada parte:

- `public`: Acessível por qualquer classe.
- `static`: Pertence à classe, não a instâncias.
- `void`: Não retorna nenhum valor.
- `main`: Nome do método.
- `String[] args`: Parâmetros do método, um array de Strings que pode receber argumentos da linha de comando.

## Exemplo de um programa Java completo

Vamos ver um exemplo que combina classe e método `main`:

```
public class OlaMundo {  
    public static void main(String[] args) {  
        System.out.println("Olá, Mundo!");  
    }  
}
```

Este programa imprime "Olá, Mundo!" no console. Para executá-lo:

1. Salve o código em um arquivo chamado `OlaMundo.java`.
2. Compile com `javac OlaMundo.java`.
3. Execute com `java OlaMundo`.

Ou use a extensão de Java do VS Code, aí basta utilizar a opção Run Java.

## Comentários em Java

Comentários são trechos de texto ignorados pelo compilador, usados para explicar o código.

**Comentário de linha única:** Inicia com `//` e vai até o fim da linha.

```
// Isto é um comentário de linha única  
int idade = 25; // A variável idade armazena a idade da pessoa
```

**Comentário de múltiplas linhas:** Inicia com `/*` e termina com `*/`.

```
/*  
 * Este é um comentário  
 * de múltiplas linhas  
*/
```

```
*/
```

**Comentário de documentação:** Inicia com `/**` e termina com `*/`. Usado para gerar documentação com o Javadoc.

```
/**
 * Classe Exemplo demonstra o uso de comentários de documentação.
 */
public class Exemplo {
    /**
     * Método principal que inicia o programa.
     * @param args Argumentos de linha de comando.
     */
    public static void main(String[] args) {
        // Código aqui
    }
}
```

## Convenções de nomenclatura

Seguir convenções de nomenclatura torna o código mais legível e padronizado.

**Classes e interfaces:** Iniciam com letra maiúscula e usam PascalCase.

```
public class CalculadoraSimples {
    // Código da classe
}

public interface OperacaoMatematica {
    // Métodos da interface
}
```

**Métodos e variáveis:** Iniciam com letra minúscula e usam camelCase.

```
public void calcularMedia() {
    // Código do método
}
```

```
}  
  
int totalAlunos;  
String nomeCompleto;
```

**Constantes:** Letras maiúsculas com palavras separadas por sublinhado (\_).

```
public static final double PI = 3.1415;  
public static final String MENSAGEM_ERRO = "Operação inválida";
```

**Pacotes:** Letras minúsculas, refletindo a estrutura do projeto.

```
package br.com.meuapp.util;
```

## Indentação e formatação

Uma boa formatação facilita a leitura e manutenção do código.

**Indentação:** Utilize 4 espaços ou um tab para cada nível de bloco.

```
public class ExemploIndentacao {  
    public void meuMetodo() {  
        if (condicao) {  
            // Código indentado  
        } else {  
            // Outro código indentado  
        }  
    }  
}
```

**Chaves:** A abertura da chave { pode ser na mesma linha ou na linha abaixo, mas mantenha consistência.

```
// Estilo K&R
public class Exemplo {
    public void metodo() {
        // Código
    }
}

// Estilo Allman
public class Exemplo
{
    public void metodo()
    {
        // Código
    }
}
```

### Boas práticas adicionais

- **Limite de caracteres por linha:** Procure não ultrapassar 80 a 120 caracteres por linha.

**Espaços em branco:** Use espaços para separar operadores e após vírgulas.

```
int soma = a + b;
System.out.println("Resultado: " + soma);
```

**Comentários claros e objetivos:** Evite comentários óbvios. Comente o porquê, não o como.

```
// Má prática
int idade = 30; // Define a idade como 30

// Boa prática
// Verifica se o usuário é maior de idade para permitir acesso
if (idade >= 18) {
    permitirAcesso();
}
```

### Exemplo completo incorporando boas práticas

```

/**
 * A classe Calculadora realiza operações matemáticas básicas.
 */
public class Calculadora {
    /**
     * Soma dois números inteiros.
     * @param a Primeiro número.
     * @param b Segundo número.
     * @return Resultado da soma.
     */
    public int somar(int a, int b) {
        return a + b;
    }

    /**
     * Subtrai o segundo número do primeiro.
     * @param a Minuendo.
     * @param b Subtraendo.
     * @return Resultado da subtração.
     */
    public int subtrair(int a, int b) {
        return a - b;
    }

    public static void main(String[] args) {
        Calculadora calc = new Calculadora();
        int resultadoSoma = calc.somar(5, 3);
        int resultadoSubtracao = calc.subtrair(10, 4);

        System.out.println("Resultado da Soma: " + resultadoSoma);
        System.out.println("Resultado da Subtração: " +
resultadoSubtracao);
    }
}

```

### Dica extra

Utilize ferramentas como o **Checkstyle** ou o **SonarLint** para analisar e garantir que seu código segue as convenções e boas práticas. Essas ferramentas ajudam a identificar problemas de formatação, nomenclatura e possíveis bugs, contribuindo para um código mais limpo e profissional.

## Capítulo 2: Tipos de dados e variáveis

### Tipos de dados primitivos



Em Java, os tipos de dados primitivos são os blocos básicos para a construção de variáveis. Eles armazenam valores simples e não são considerados objetos. Existem oito tipos de dados primitivos:

#### **Inteiros:**

- byte: 8 bits, valores de -128 a 127.
- short: 16 bits, valores de -32.768 a 32.767.
- int: 32 bits, valores de  $-2^{31}$  a  $2^{31} - 1$ .
- long: 64 bits, valores de  $-2^{63}$  a  $2^{63} - 1$ .

#### **Exemplo:**

```
byte idade = 25;
short ano = 2021;
int populacao = 1500000;
long distancia = 15000000000L; // O 'L' indica um literal do tipo
```

#### **Ponto flutuante:**

- float: 32 bits, precisão simples.
- double: 64 bits, precisão dupla.

#### **Exemplo:**

```
float altura = 1.75f; // O 'f' indica um literal do tipo float
double peso = 70.5;
```

#### **Caractere:**

- char: 16 bits Unicode, representa um único caractere.

#### **Exemplo:**

```
char letra = 'A';
char simbolo = '#';
```

## Booleano:

- `boolean`: representa valores lógicos `true` (verdadeiro) ou `false` (falso).

## Exemplo:

```
boolean ativo = true;  
boolean aprovado = false;
```

## Variáveis e constantes

As variáveis são utilizadas para armazenar dados que podem mudar durante a execução do programa. Já as constantes mantêm um valor fixo após serem inicializadas.

### Declaração e inicialização

**Declaração:** Especifica o tipo e o nome da variável.

```
int numero; // Declara a variável 'numero' do tipo int
```

**Inicialização:** Atribui um valor inicial à variável.

```
numero = 10; // Atribui o valor 10 à variável 'numero'
```

### Declaração e inicialização simultânea:

```
int contador = 0;
```

## Constantes

Para declarar uma constante, utiliza-se a palavra-chave `final`. É uma boa prática nomear constantes em letras maiúsculas.

```
final double PI = 3.1415;  
final int MAX_USUARIOS = 100;
```

## Escopo das variáveis

O escopo determina onde a variável é acessível dentro do código.

**Variáveis de instância:** Declaradas dentro da classe, mas fora de qualquer método. Acessíveis por todos os métodos da classe.

```
public class Pessoa {  
    String nome; // Variável de instância  
  
    public void exibirNome() {  
        System.out.println(nome);  
    }  
}
```

**Variáveis locais:** Declaradas dentro de métodos, construtores ou blocos. Só podem ser usadas dentro desse escopo.

```
public void calcularIdade() {  
    int idade = 30; // Variável local  
    System.out.println("Idade: " + idade);  
}
```

**Variáveis de classe (estáticas):** Declaradas com o modificador static. Pertencem à classe, não a instâncias específicas.

```
public class Configuracao {  
    public static String sistemaOperacional = "Windows";  
}
```

## Conversão de tipos

A conversão de tipos (casting) permite transformar um valor de um tipo de dado em outro. Pode ser implícita ou explícita.

## Conversão implícita (widening)

Ocorre automaticamente quando se atribui um tipo de menor capacidade a um de maior capacidade.

```
int numeroInt = 100;
long numeroLong = numeroInt; // Conversão implícita de int para long
double numeroDouble = numeroLong; // Conversão implícita de long para double
```

## Conversão explícita (narrowing)

Necessita de casting explícito quando se atribui um tipo de maior capacidade a um de menor capacidade.

```
double numeroDouble = 9.78;
int numeroInt = (int) numeroDouble; // Conversão explícita de double para int
System.out.println(numeroInt); // Saída: 9 (perde a parte decimal)
```

**Atenção:** A conversão explícita pode resultar em perda de dados ou precisão.

## Exemplos de conversão

**De int para byte (pode causar overflow):**

```
int numeroInt = 130;
byte numeroByte = (byte) numeroInt;
System.out.println(numeroByte); // Saída: -126 (valor incorreto devido ao overflow)
```

**De String para número:**

```
String valorString = "50";
int valorInt = Integer.parseInt(valorString);
double valorDouble = Double.parseDouble("3.1415");
```

## De número para String:

```
int numero = 100;  
String texto = Integer.toString(numero);
```

## Classes Wrapper

Para cada tipo primitivo, existe uma classe correspondente na linguagem Java. Elas são úteis quando precisamos tratar tipos primitivos como objetos.

| Tipo primitivo | Classe Wrapper |
|----------------|----------------|
| byte           | Byte           |
| short          | Short          |
| int            | Integer        |
| long           | Long           |
| float          | Float          |
| double         | Double         |
| char           | Character      |
| boolean        | Boolean        |

## Exemplo de uso das classes Wrapper

```
Integer numeroObjeto = Integer.valueOf(100); // Autoboxing  
int numeroPrimitivo = numeroObjeto.intValue(); // Unboxing
```

## Autoboxing e Unboxing

**Autoboxing:** Conversão automática de tipo primitivo para objeto wrapper.

```
Integer numObjeto = 50; // Autoboxing de int para Integer
```

**Unboxing:** Conversão automática de objeto wrapper para tipo primitivo.

```
int numPrimitivo = numObjeto; // Unboxing de Integer para int
```

### Exemplo completo

```
public class ExemploVariaveis {
    public static void main(String[] args) {
        // Tipos primitivos
        int idade = 25;
        double salario = 3500.75;
        char genero = 'M';
        boolean empregado = true;

        // Constantes
        final double PI = 3.1415;

        // Conversão implícita
        long idadeLong = idade;
        float salarioFloat = (float) salario; // Necessita casting
        explícito

        // Conversão explícita
        int valor = (int) 9.99; // Resultado: 9

        // Uso de classes wrapper
        Integer idadeObjeto = Integer.valueOf(idade);
        int idadePrimitiva = idadeObjeto.intValue();

        // Exibindo os valores
        System.out.println("Idade: " + idade);
        System.out.println("Salário: " + salario);
        System.out.println("Gênero: " + genero);
        System.out.println("Empregado: " + empregado);
        System.out.println("PI: " + PI);
        System.out.println("Idade em long: " + idadeLong);
        System.out.println("Salário em float: " + salarioFloat);
        System.out.println("Valor convertido: " + valor);
    }
}
```

### Dicas práticas

**Nomenclatura clara:** Use nomes significativos para variáveis, facilitando a compreensão do código.

```
double taxaJuros = 0.05;
int quantidadeItens = 10;
```

- 
- **Inicialização:** Sempre inicialize suas variáveis para evitar comportamentos inesperados.
- **Escopo:** Esteja atento ao escopo das variáveis para evitar erros de acesso.
- **Conversão segura:** Ao converter tipos, certifique-se de que o valor está dentro do intervalo permitido pelo tipo de destino.

## Capítulo 3: Operadores e expressões

Neste capítulo, exploraremos os operadores disponíveis em Java e como usá-los para construir expressões e manipular dados. Os operadores são símbolos que instruem o compilador a realizar operações específicas, como aritmética, lógica ou atribuição. Compreender como utilizá-los é fundamental para escrever código eficaz e eficiente.

### Operadores aritméticos

Os operadores aritméticos são usados para realizar operações matemáticas básicas.

| Operador | Descrição      | Exemplo |
|----------|----------------|---------|
| +        | Adição         | a + b   |
| -        | Subtração      | a - b   |
| *        | Multiplicação  | a * b   |
| /        | Divisão        | a / b   |
| %        | Módulo (resto) | a % b   |

### Exemplos de uso:

```
public class OperadoresAritmeticos {
    public static void main(String[] args) {
        int a = 10;
```

```

    int b = 3;

    int soma = a + b;           // Resultado: 13
    int subtracao = a - b;      // Resultado: 7
    int multiplicacao = a * b;  // Resultado: 30
    int divisao = a / b;        // Resultado: 3 (divisão inteira)
    int modulo = a % b;         // Resultado: 1 (resto da divisão)

    System.out.println("Soma: " + soma);
    System.out.println("Subtração: " + subtracao);
    System.out.println("Multiplicação: " + multiplicacao);
    System.out.println("Divisão: " + divisao);
    System.out.println("Módulo: " + modulo);
}
}

```

#### Observação sobre divisão:

- Quando ambos os operandos são inteiros, a divisão resulta em um inteiro, descartando a parte decimal.
- Para obter um resultado com casas decimais, pelo menos um dos operandos deve ser do tipo double ou float.

#### Exemplo de divisão com números reais:

```

double x = 10;
double y = 3;
double resultado = x / y; // Resultado: 3.3333333333333335

```

### Operadores relacionais e lógicos

Esses operadores são usados para comparar valores e combinar expressões lógicas.

#### Operadores relacionais:

| Operador | Descrição    | Exemplo |
|----------|--------------|---------|
| ==       | Igual a      | a == b  |
| !=       | Diferente de | a != b  |
| >        | Maior que    | a > b   |



|    |                  |        |
|----|------------------|--------|
| <  | Menor que        | a < b  |
| >= | Maior ou igual a | a >= b |
| <= | Menor ou igual a | a <= b |

### Operadores lógicos:

| Operador | Descrição  | Exemplo        |
|----------|------------|----------------|
| &&       | AND lógico | cond1 && cond2 |
|          | OR lógico  | cond1    cond2 |
| !        | NOT lógico | !cond          |

### Exemplos de uso:

```
public class OperadoresRelacionaisLogicos {
    public static void main(String[] args) {
        int idade = 20;
        boolean maiorDeIdade = idade >= 18;
        boolean menorDeIdade = idade < 18;

        System.out.println("Maior de idade: " + maiorDeIdade); // true
        System.out.println("Menor de idade: " + menorDeIdade); // false

        boolean condicao1 = true;
        boolean condicao2 = false;

        boolean resultadoAND = condicao1 && condicao2; // false
        boolean resultadoOR = condicao1 || condicao2; // true
        boolean resultadoNOT = !condicao1;           // false

        System.out.println("Resultado AND: " + resultadoAND);
        System.out.println("Resultado OR: " + resultadoOR);
        System.out.println("Resultado NOT: " + resultadoNOT);
    }
}
```

### Curto-circuito nos operadores lógicos:

- **&& (AND lógico):** Se a primeira condição for falsa, a segunda não é avaliada, pois o resultado será necessariamente falso.

- **|| (OR lógico):** Se a primeira condição for verdadeira, a segunda não é avaliada, pois o resultado será necessariamente verdadeiro.

#### Exemplo demonstrando curto-circuito:

```
public class CurtoCircuito {
    public static void main(String[] args) {
        int a = 0;

        if (a != 0 && (10 / a) > 1) {
            System.out.println("Não será executado devido ao
curto-circuito.");
        } else {
            System.out.println("Evita divisão por zero.");
        }
    }
}
```

#### Operadores de atribuição e incremento

##### Operadores de atribuição:

O operador de atribuição simples é o =.

```
int x = 10;
```

##### Operadores de atribuição composta:

| Operador | Descrição                  | Exemplo | Equivalente a |
|----------|----------------------------|---------|---------------|
| +=       | Adição e atribuição        | a += b  | a = a + b     |
| -=       | Subtração e atribuição     | a -= b  | a = a - b     |
| *=       | Multiplicação e atribuição | a *= b  | a = a * b     |
| /=       | Divisão e atribuição       | a /= b  | a = a / b     |
| %=       | Módulo e atribuição        | a %= b  | a = a % b     |

##### Exemplo:

```
public class OperadoresAtribuicao {
    public static void main(String[] args) {
        int a = 5;
        a += 3; // a = a + 3 -> a = 8
        a *= 2; // a = a * 2 -> a = 16

        System.out.println("Valor de a: " + a); // 16
    }
}
```

### Operadores de incremento e decremento:

| Operador | Descrição       | Exemplo    |
|----------|-----------------|------------|
| ++       | Incrementa em 1 | a++ ou ++a |
| --       | Decrementa em 1 | a-- ou --a |

### Pré-incremento vs. Pós-incremento:

- **Pré-incremento (++a):** Incrementa o valor antes de usá-lo na expressão.
- **Pós-incremento (a++):** Usa o valor atual na expressão e depois incrementa.

### Exemplo:

```
public class IncrementoDecremento {
    public static void main(String[] args) {
        int a = 5;
        int b = ++a; // a = 6, b = 6 (pré-incremento)
        int c = a++; // c = 6, a = 7 (pós-incremento)

        System.out.println("a: " + a); // 7
        System.out.println("b: " + b); // 6
        System.out.println("c: " + c); // 6
    }
}
```

---

### Precedência de operadores

A precedência determina a ordem em que os operadores são avaliados em uma expressão.

### Ordem de precedência (da maior para a menor):

1. **Operadores unários:** ++, --, + (unário), - (unário), !
2. **Multiplicativos:** \*, /, %
3. **Aditivos:** +, -
4. **Relacionais:** <, >, <=, >=
5. **Igualdade:** ==, !=
6. **AND lógico:** &&
7. **OR lógico:** ||
8. **Atribuição:** =, +=, -=, \*=, /=, %=

**Exemplo de expressão complexa:**

```
int resultado = 10 + 2 * 5; // resultado = 10 + (2 * 5) = 20
```

**Uso de parênteses para controlar a precedência:**

```
int resultado = (10 + 2) * 5; // resultado = (12) * 5 = 60
```

**Exemplo completo combinando operadores:**

```
public class PrecedenciaOperadores {
    public static void main(String[] args) {
        int a = 10;
        int b = 5;
        int c = 2;

        int resultado1 = a + b * c;           // 10 + (5 * 2) = 20
        int resultado2 = (a + b) * c;         // (10 + 5) * 2 = 30
        int resultado3 = a / b / c;           // (10 / 5) / 2 = 1
        int resultado4 = a / (b / c);         // 10 / (5 / 2) = 10 / 2 = 5

        System.out.println("Resultado 1: " + resultado1);
        System.out.println("Resultado 2: " + resultado2);
        System.out.println("Resultado 3: " + resultado3);
        System.out.println("Resultado 4: " + resultado4);
    }
}
```

---

## Expressões condicionais e ternário

O operador ternário é uma forma concisa de expressar uma condição.

### Sintaxe:

```
variavel = (condicao) ? valorSeVerdadeiro : valorSeFalso;
```

### Exemplo:

```
public class OperadorTernario {  
    public static void main(String[] args) {  
        int nota = 75;  
        String resultado = (nota >= 60) ? "Aprovado" : "Reprovado";  
        System.out.println("Resultado: " + resultado); // Aprovado  
    }  
}
```

---

## Operadores bit a bit (bitwise)

Embora menos comuns em programação de alto nível, os operadores bit a bit são úteis em situações específicas.

| Operador | Descrição                 | Exemplo |
|----------|---------------------------|---------|
| &        | AND bit a bit             | a & b   |
|          | OR bit a bit              | a   b   |
| ^        | XOR bit a bit             | a ^ b   |
| ~        | NOT bit a bit             | ~a      |
| <<       | Shift à esquerda          | a << n  |
| >>       | Shift à direita           | a >> n  |
| >>>      | Shift à direita sem sinal | a >>> n |

### Exemplo de uso:

```

public class OperadoresBitwise {
    public static void main(String[] args) {
        int a = 5;    // 0101 em binário
        int b = 3;    // 0011 em binário

        int and = a & b;    // 0101 & 0011 = 0001 (1)
        int or = a | b;    // 0101 | 0011 = 0111 (7)
        int xor = a ^ b;    // 0101 ^ 0011 = 0110 (6)
        int notA = ~a;    // ~0101 = 1010 (complemento de dois)

        System.out.println("AND: " + and);
        System.out.println("OR: " + or);
        System.out.println("XOR: " + xor);
        System.out.println("NOT A: " + notA);
    }
}

```

---

## Conversão de tipos em expressões

Ao combinar diferentes tipos de dados em expressões, o Java realiza promoções de tipo para evitar perda de precisão.

- **Inteiros menores que `int` são promovidos a `int`:** `byte`, `short`, `char` são convertidos para `int` ao participar de expressões.
- **Se houver `double` na expressão, o resultado será `double`.**

**Exemplo:**

```

byte b = 10;
byte c = 20;
int resultado = b + c; // b e c são promovidos a int

double x = 5.5;
int y = 10;
double z = x + y; // y é promovido a double

```

---

## Resumo e boas práticas

- Use parênteses para tornar expressões complexas mais claras.
- Esteja atento à precedência dos operadores para evitar resultados inesperados.

- Escolha operadores adequados para a situação, preferindo clareza sobre concisão.
  - Ao usar operadores de incremento/decremento, seja cuidadoso com a posição (pré ou pós) e os efeitos colaterais.
- 

### Exercícios práticos

1. **Calculadora simples:** Crie um programa que recebe dois números e um operador aritmético do usuário e exibe o resultado da operação.
2. **Verificação de maioridade:** Escreva um programa que verifica se uma pessoa é maior de idade com base na idade fornecida.
3. **Número par ou ímpar:** Desenvolva um código que determina se um número é par ou ímpar usando o operador módulo.

### Exemplo do exercício 1:

```
import java.util.Scanner;

public class CalculadoraSimples {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.println("Digite o primeiro número:");
        double num1 = scanner.nextDouble();

        System.out.println("Digite o operador (+, -, *, /):");
        char operador = scanner.next().charAt(0);

        System.out.println("Digite o segundo número:");
        double num2 = scanner.nextDouble();

        double resultado = 0;
        boolean operacaoValida = true;

        switch (operador) {
            case '+':
                resultado = num1 + num2;
                break;
            case '-':
                resultado = num1 - num2;
                break;
            case '*':
                resultado = num1 * num2;
                break;
        }
    }
}
```

```

        case '/':
            if (num2 != 0) {
                resultado = num1 / num2;
            } else {
                System.out.println("Erro: Divisão por zero!");
                operacaoValida = false;
            }
            break;
        default:
            System.out.println("Operador inválido!");
            operacaoValida = false;
    }

    if (operacaoValida) {
        System.out.println("Resultado: " + resultado);
    }

    scanner.close();
}
}

```

## Capítulo 4: Estruturas de controle de fluxo

As estruturas de controle de fluxo em Java permitem que você direcione a execução do programa com base em condições e repita ações conforme necessário. Elas são fundamentais para criar programas dinâmicos e responsivos, que podem tomar decisões e executar tarefas repetitivas de forma eficiente.

---

### Estruturas condicionais

As estruturas condicionais permitem que o programa tome decisões com base em condições booleanas (verdadeiro ou falso). As principais estruturas condicionais em Java são `if`, `else if`, `else` e `switch`.

#### Uso do `if`, `else if` e `else`

O comando `if` avalia uma expressão booleana e executa um bloco de código se a condição for verdadeira. O `else if` e o `else` fornecem caminhos alternativos quando a condição inicial não é satisfeita.

#### Sintaxe básica:



```
if (condicao1) {
    // Bloco executado se condicao1 for verdadeira
} else if (condicao2) {
    // Bloco executado se condicao2 for verdadeira
} else {
    // Bloco executado se nenhuma das condições anteriores for verdadeira
}
```

### Exemplo prático:

```
public class AvaliacaoDesempenho {
    public static void main(String[] args) {
        int nota = 85;

        if (nota >= 90) {
            System.out.println("Excelente");
        } else if (nota >= 70) {
            System.out.println("Bom");
        } else if (nota >= 50) {
            System.out.println("Regular");
        } else {
            System.out.println("Insuficiente");
        }
    }
}
```

Neste exemplo, o programa avalia a variável `nota` e imprime uma mensagem correspondente ao desempenho.

### Uso do switch

O `switch` é uma estrutura condicional que compara o valor de uma expressão com múltiplas constantes e executa o bloco correspondente.

### Sintaxe básica:

```
switch (expressao) {
    case valor1:
        // Bloco executado se expressao == valor1
        break;
    case valor2:
```

```
    // Bloco executado se expressao == valor2
    break;
// Outros casos
default:
    // Bloco executado se nenhum caso for correspondente
}
```

### Exemplo prático:

```
public class DiaDaSemana {
    public static void main(String[] args) {
        int dia = 3;
        String nomeDia;

        switch (dia) {
            case 1:
                nomeDia = "Domingo";
                break;
            case 2:
                nomeDia = "Segunda-feira";
                break;
            case 3:
                nomeDia = "Terça-feira";
                break;
            case 4:
                nomeDia = "Quarta-feira";
                break;
            case 5:
                nomeDia = "Quinta-feira";
                break;
            case 6:
                nomeDia = "Sexta-feira";
                break;
            case 7:
                nomeDia = "Sábado";
                break;
            default:
                nomeDia = "Dia inválido";
                break;
        }

        System.out.println("Dia da semana: " + nomeDia);
    }
}
```

Neste exemplo, o programa associa um número de 1 a 7 a um dia da semana.

### Observações sobre o switch:

- A expressão no switch deve ser de um tipo compatível: byte, short, int, char, String ou enumerações.
  - O break é usado para sair do switch após executar o caso correspondente. Se omitido, o fluxo continua para o próximo caso (comportamento conhecido como "fall-through").
  - O default é opcional e executado se nenhum caso corresponder.
- 

### Estruturas de repetição

As estruturas de repetição (loops) permitem executar um bloco de código múltiplas vezes, seja por um número determinado ou enquanto uma condição for satisfeita. Java oferece três tipos principais de loops: for, while e do-while.

#### Loop for

Usado quando se sabe o número exato de iterações.

#### Sintaxe básica:

```
for (inicializacao; condicao; incremento) {  
    // Bloco de código a ser repetido  
}
```

#### Exemplo prático:

```
public class ContagemRegressiva {  
    public static void main(String[] args) {  
        for (int i = 10; i >= 0; i--) {  
            System.out.println("Contagem: " + i);  
        }  
        System.out.println("Lançar!");  
    }  
}
```

Este programa realiza uma contagem regressiva de 10 a 0.

## Loop while

Usado quando não se sabe exatamente quantas vezes o loop será executado, mas depende de uma condição.

### Sintaxe básica:

```
while (condicao) {  
    // Bloco de código a ser repetido enquanto a condicao for verdadeira  
}
```

### Exemplo prático:

```
import java.util.Scanner;  
  
public class LeituraSenha {  
    public static void main(String[] args) {  
        Scanner scanner = new Scanner(System.in);  
        String senha;  
  
        do {  
            System.out.print("Digite a senha: ");  
            senha = scanner.nextLine();  
        } while (!senha.equals("1234"));  
  
        System.out.println("Senha correta!");  
        scanner.close();  
    }  
}
```

## Loop do-while

Semelhante ao while, mas garante que o bloco de código seja executado pelo menos uma vez, já que a condição é verificada após a execução.

### Sintaxe básica:

```
do {  
    // Bloco de código a ser repetido  
} while (condicao);
```

## Exemplo prático:

```
public class MenuSimples {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int opcao;

        do {
            System.out.println("Menu:");
            System.out.println("1 - Opção A");
            System.out.println("2 - Opção B");
            System.out.println("0 - Sair");
            System.out.print("Escolha uma opção: ");
            opcao = scanner.nextInt();

            switch (opcao) {
                case 1:
                    System.out.println("Você escolheu a Opção A");
                    break;
                case 2:
                    System.out.println("Você escolheu a Opção B");
                    break;
                case 0:
                    System.out.println("Saindo...");
                    break;
                default:
                    System.out.println("Opção inválida");
                    break;
            }
        } while (opcao != 0);

        scanner.close();
    }
}
```

---

## Controle de loops

Às vezes, é necessário alterar o fluxo normal de execução dentro de um loop. As palavras-chave `break` e `continue` são usadas para esse fim.

### Uso do `break`

Interrompe a execução do loop imediatamente, saindo dele.

### Exemplo prático:

```
public class BuscaNumero {
    public static void main(String[] args) {
        int[] numeros = {1, 3, 5, 7, 9, 11};
        int numeroProcurado = 7;

        for (int num : numeros) {
            if (num == numeroProcurado) {
                System.out.println("Número encontrado: " + num);
                break; // Sai do loop ao encontrar o número
            }
        }
    }
}
```

### Uso do continue

Interrompe a iteração atual e passa para a próxima iteração do loop.

### Exemplo prático:

```
public class NumerosPares {
    public static void main(String[] args) {
        for (int i = 0; i <= 10; i++) {
            if (i % 2 != 0) {
                continue; // Se o número for ímpar, pula para a próxima
                iteração
            }
            System.out.println("Número par: " + i);
        }
    }
}
```

Este programa imprime apenas números pares entre 0 e 10.

### Rotulando loops

Em situações com loops aninhados, é possível rotular loops para controlar quais loops devem ser afetados pelo break ou continue.

### Sintaxe de rótulos:

```
nomeDoRotulo:
for (inicializacao; condicao; incremento) {
    // Bloco de código
}
```

### Exemplo prático:

```
public class BuscaMatriz {
    public static void main(String[] args) {
        int[][] matriz = {
            {1, 2, 3},
            {4, 5, 6},
            {7, 8, 9}
        };
        int numeroProcurado = 5;
        boolean encontrado = false;

        buscarNumero:
        for (int i = 0; i < matriz.length; i++) {
            for (int j = 0; j < matriz[i].length; j++) {
                if (matriz[i][j] == numeroProcurado) {
                    System.out.println("Número encontrado na posição ["
+ i + "][" + j + "]");
                    encontrado = true;
                    break buscarNumero; // Sai dos dois loops
                }
            }
        }

        if (!encontrado) {
            System.out.println("Número não encontrado");
        }
    }
}
```

Neste exemplo, ao encontrar o número procurado, o programa usa `break buscarNumero;` para sair de ambos os loops.

### Boas práticas com estruturas de controle de fluxo

- **Evite loops infinitos:** Certifique-se de que a condição do loop será eventualmente falsa ou que existe um `break` adequado.

- **Use break e continue com parcimônia:** Embora úteis, seu uso excessivo pode tornar o código difícil de entender. Prefira estruturas claras e legíveis.
- **Indentação correta:** Mantenha o código bem formatado para facilitar a leitura e manutenção.
- **Condições claras:** Escreva condições que sejam fáceis de entender e evite negações múltiplas.

## Capítulo 5: Arrays e coleções

Neste capítulo, exploraremos como armazenar e manipular conjuntos de dados em Java utilizando arrays e coleções. Esses recursos são fundamentais para lidar com múltiplos valores de forma eficiente e organizada, permitindo que você desenvolva programas mais robustos e flexíveis.

---

### Arrays unidimensionais

Um array é uma estrutura de dados que armazena uma coleção de elementos do mesmo tipo em posições contíguas na memória. Em Java, os arrays são objetos que facilitam o armazenamento e o acesso a múltiplos valores usando um único nome de variável e índices numéricos.

### Declaração e inicialização

Existem várias maneiras de declarar e inicializar um array em Java.

#### Declaração e alocação separadas:

```
int[] numeros;           // Declaração
numeros = new int[5];    // Alocação de espaço para 5 elementos
```

#### Declaração e alocação combinadas:

```
int[] numeros = new int[5];
```

#### Inicialização com valores conhecidos:

```
int[] numeros = {10, 20, 30, 40, 50};
```



## Acesso aos elementos

Os elementos de um array são acessados por índices, que começam em zero.

### Exemplo de acesso e modificação:

```
public class ExemploArray {  
    public static void main(String[] args) {  
        int[] numeros = new int[5];  
  
        // Atribuindo valores ao array  
        numeros[0] = 5;  
        numeros[1] = 10;  
        numeros[2] = 15;  
        numeros[3] = 20;  
        numeros[4] = 25;  
  
        // Acessando e imprimindo os valores  
        for (int i = 0; i < numeros.length; i++) {  
            System.out.println("Elemento na posição " + i + ": " +  
numeros[i]);  
        }  
    }  
}
```

### Saída:

```
Elemento na posição 0: 5  
Elemento na posição 1: 10  
Elemento na posição 2: 15  
Elemento na posição 3: 20  
Elemento na posição 4: 25
```

## Propriedade length

O atributo length de um array retorna seu tamanho (número de elementos).

```
System.out.println("Tamanho do array: " + numeros.length);
```

## Percorrendo arrays com o loop for-each

O loop for-each simplifica a iteração sobre arrays.

### Exemplo:

```
public class ExemploForEach {  
    public static void main(String[] args) {  
        String[] nomes = {"Ana", "Bruno", "Carla", "Daniel"};  
  
        for (String nome : nomes) {  
            System.out.println("Nome: " + nome);  
        }  
    }  
}
```

### Saída:

```
Nome: Ana  
Nome: Bruno  
Nome: Carla  
Nome: Daniel
```

---

## Arrays multidimensionais

Arrays multidimensionais são arrays que contêm outros arrays como elementos. Em Java, o tipo mais comum é o array bidimensional (matriz), que representa uma tabela de valores.

### Declaração e inicialização

#### Array bidimensional:

```
int[][] matriz = new int[3][4]; // 3 linhas e 4 colunas
```

#### Inicialização com valores conhecidos:

```
int[][] matriz = {
```

```
{1, 2, 3},  
{4, 5, 6},  
{7, 8, 9}  
};
```

### Acesso aos elementos

Os elementos são acessados usando dois índices: o primeiro para a linha e o segundo para a coluna.

### Exemplo de acesso e modificação:

```
public class ExemploMatriz {  
    public static void main(String[] args) {  
        int[][] matriz = new int[2][3];  
  
        // Atribuindo valores à matriz  
        matriz[0][0] = 10;  
        matriz[0][1] = 20;  
        matriz[0][2] = 30;  
        matriz[1][0] = 40;  
        matriz[1][1] = 50;  
        matriz[1][2] = 60;  
  
        // Acessando e imprimindo os valores  
        for (int i = 0; i < matriz.length; i++) {  
            for (int j = 0; j < matriz[i].length; j++) {  
                System.out.print(matriz[i][j] + "\t");  
            }  
            System.out.println();  
        }  
    }  
}
```

### Saída:

```
10    20    30  
40    50    60
```

### Arrays irregulares (jagged arrays)

Em Java, é possível criar arrays multidimensionais com linhas de tamanhos diferentes.

### Exemplo:

```

public class ExemploArrayIrregular {
    public static void main(String[] args) {
        int[][] irregular = new int[3][]; // Define um array com 3
linhas

        irregular[0] = new int[2]; // Primeira linha com 2 colunas
        irregular[1] = new int[4]; // Segunda linha com 4 colunas
        irregular[2] = new int[3]; // Terceira linha com 3 colunas

        // Atribuindo valores e imprimindo
        int valor = 1;
        for (int i = 0; i < irregular.length; i++) {
            for (int j = 0; j < irregular[i].length; j++) {
                irregular[i][j] = valor++;
                System.out.print(irregular[i][j] + "\t");
            }
            System.out.println();
        }
    }
}

```

Saída:

```

1      2
3      4      5      6
7      8      9

```

---

## Introdução às coleções

As coleções em Java são estruturas de dados que permitem armazenar e manipular grupos de objetos de forma dinâmica. Elas fazem parte do **Framework Collections**, que oferece diversas interfaces e classes prontas para uso, facilitando operações como inserção, remoção, busca e ordenação.

### Principais interfaces:

1. **List**: Representa uma coleção ordenada que permite elementos duplicados.
2. **Set**: Representa uma coleção que não permite elementos duplicados.
3. **Map**: Armazena pares chave-valor, não permite chaves duplicadas.

### Utilizando a interface List

A interface List e suas implementações, como ArrayList e LinkedList, permitem manipular listas de elementos.

### Exemplo com ArrayList:

```
import java.util.ArrayList;
import java.util.List;

public class ExemploList {
    public static void main(String[] args) {
        List<String> frutas = new ArrayList<>();

        // Adicionando elementos
        frutas.add("Maçã");
        frutas.add("Banana");
        frutas.add("Laranja");

        // Acessando elementos
        System.out.println("Primeira fruta: " + frutas.get(0));

        // Iterando sobre a lista
        for (String fruta : frutas) {
            System.out.println("Fruta: " + fruta);
        }

        // Verificando se um elemento existe
        if (frutas.contains("Banana")) {
            System.out.println("A lista contém Banana");
        }

        // Removendo um elemento
        frutas.remove("Laranja");
        System.out.println("Lista após remoção: " + frutas);
    }
}
```

### Saída:

```
Primeira fruta: Maçã
Fruta: Maçã
Fruta: Banana
Fruta: Laranja
A lista contém Banana
```

Lista após remoção: [Maçã, Banana]

### Utilizando a interface Set

O Set é usado quando não se deseja elementos duplicados. Implementações comuns incluem HashSet e TreeSet.

### Exemplo com HashSet:

```
import java.util.HashSet;
import java.util.Set;

public class ExemploSet {
    public static void main(String[] args) {
        Set<Integer> numeros = new HashSet<>();

        // Adicionando elementos
        numeros.add(10);
        numeros.add(20);
        numeros.add(30);
        numeros.add(20); // Tentativa de adicionar elemento duplicado

        // Iterando sobre o conjunto
        for (int num : numeros) {
            System.out.println("Número: " + num);
        }

        // Verificando tamanho
        System.out.println("Tamanho do conjunto: " + numeros.size());
    }
}
```

### Saída (a ordem pode variar):

```
Número: 20
Número: 10
Número: 30
Tamanho do conjunto: 3
```

**Observação:** O HashSet não mantém a ordem de inserção.

## Utilizando a interface Map

O Map armazena pares chave-valor, permitindo associar uma chave única a um valor.

### Exemplo com HashMap:

```
import java.util.HashMap;
import java.util.Map;

public class ExemploMap {
    public static void main(String[] args) {
        Map<String, String> capitais = new HashMap<>();

        // Adicionando pares chave-valor
        capitais.put("Brasil", "Brasília");
        capitais.put("França", "Paris");
        capitais.put("Japão", "Tóquio");

        // Acessando um valor pela chave
        System.out.println("Capital do Brasil: " +
            capitais.get("Brasil"));

        // Iterando sobre o mapa
        for (Map.Entry<String, String> entrada : capitais.entrySet()) {
            System.out.println("País: " + entrada.getKey() + " -
            Capital: " + entrada.getValue());
        }

        // Verificando se uma chave existe
        if (capitais.containsKey("França")) {
            System.out.println("O mapa contém a chave 'França'");
        }

        // Removendo uma entrada
        capitais.remove("Japão");
        System.out.println("Mapa após remoção: " + capitais);
    }
}
```

### Saída:

```
Capital do Brasil: Brasília
País: Brasil - Capital: Brasília
```

```
País: França - Capital: Paris
País: Japão - Capital: Tóquio
O mapa contém a chave 'França'
Mapa após remoção: {Brasil=Brasília, França=Paris}
```

---

## Generics em coleções

As coleções em Java utilizam **Generics** para especificar o tipo de elementos que armazenam, aumentando a segurança e evitando erros de tipo em tempo de execução.

### Exemplo sem Generics (Java 5 ou anterior):

```
List lista = new ArrayList();
lista.add("Texto");
lista.add(123); // Permite adicionar qualquer tipo de objeto

// Pode causar erros em tempo de execução
String elemento = (String) lista.get(1); // ClassCastException
```

### Exemplo com Generics:

```
List<String> lista = new ArrayList<>();
lista.add("Texto");
// lista.add(123); // Erro de compilação: não permite adicionar Integer

String elemento = lista.get(0); // Não é necessário casting
```

---

## Comparação entre Arrays e Coleções

| Aspecto            | Arrays                     | Coleções                                    |
|--------------------|----------------------------|---------------------------------------------|
| Tamanho            | Fixo após a criação        | Dinâmico, pode crescer ou diminuir          |
| Tipo de dados      | Tipos primitivos e objetos | Apenas objetos (autoboxing para primitivos) |
| Métodos auxiliares | Limitados                  | Ricos em métodos para manipulação           |
| Iteração           | Uso de índices             | Iteradores e loops aprimorados              |



|               |                                           |                                  |
|---------------|-------------------------------------------|----------------------------------|
| Flexibilidade | Menos flexível, mais eficiente em memória | Mais flexível, facilidade de uso |
|---------------|-------------------------------------------|----------------------------------|

### Exercícios práticos

1. **Cadastro de produtos:** Crie um programa que armazene informações de produtos (nome, preço, quantidade) em uma lista e permita ao usuário adicionar, listar e remover produtos.
2. **Contador de palavras:** Desenvolva um programa que conte a frequência de cada palavra em um texto fornecido, utilizando um Map.
3. **Conjunto de números únicos:** Escreva um programa que receba números inteiros do usuário até que ele digite zero, armazenando apenas os números únicos em um Set e, ao final, exiba-os em ordem crescente.

### Exemplo do exercício 2:

```
import java.util.HashMap;
import java.util.Map;
import java.util.Scanner;

public class ContadorPalavras {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.println("Digite um texto:");
        String texto = scanner.nextLine();

        String[] palavras = texto.toLowerCase().split("\\W+");
        Map<String, Integer> frequencia = new HashMap<>();

        for (String palavra : palavras) {
            if (frequencia.containsKey(palavra)) {
                frequencia.put(palavra, frequencia.get(palavra) + 1);
            } else {
                frequencia.put(palavra, 1);
            }
        }

        System.out.println("Frequência das palavras:");
        for (Map.Entry<String, Integer> entrada : frequencia.entrySet())
        {
            System.out.println(entrada.getKey() + ": " +
            entrada.getValue());
        }
    }
}
```

```
        scanner.close();  
    }  
}
```

**Entrada:**

```
Este é um exemplo. Este exemplo é simples.
```

**Saída:**

```
Frequência das palavras:  
este: 2  
é: 2  
um: 1  
exemplo: 2  
simples: 1
```

## Capítulo 6: Fundamentos da Programação Orientada a Objetos

A Programação Orientada a Objetos (POO) é um paradigma de programação que utiliza "objetos" e seus "relacionamentos" para desenvolver software modular, reutilizável e mais fácil de manter. Java é uma linguagem fortemente orientada a objetos, e entender seus fundamentos é essencial para aproveitar todo o potencial que ela oferece.

---

### Conceitos básicos: Classes, objetos, atributos e métodos

#### Classes e objetos

- **Classe:** É um molde ou estrutura que define atributos (propriedades) e métodos (comportamentos) comuns a um determinado tipo de objeto.
- **Objeto:** É uma instância de uma classe. É a manifestação concreta da classe na memória, com valores específicos para seus atributos.

**Exemplo de uma classe simples:**

```
public class Pessoa {  
    // Atributos  
    String nome;
```

```
int idade;

// Métodos
void apresentar() {
    System.out.println("Olá, meu nome é " + nome + " e tenho " +
idade + " anos.");
}
}
```

### Criando objetos (instâncias):

```
public class Principal {
    public static void main(String[] args) {
        // Criando um objeto da classe Pessoa
        Pessoa pessoa1 = new Pessoa();
        pessoa1.nome = "Carlos";
        pessoa1.idade = 30;
        pessoa1.apresentar();

        Pessoa pessoa2 = new Pessoa();
        pessoa2.nome = "Mariana";
        pessoa2.idade = 25;
        pessoa2.apresentar();
    }
}
```

### Saída:

```
Olá, meu nome é Carlos e tenho 30 anos.
Olá, meu nome é Mariana e tenho 25 anos.
```

### Atributos e métodos

- **Atributos:** São variáveis que representam as características de um objeto. No exemplo, nome e idade são atributos da classe Pessoa.
  - **Métodos:** São funções que definem os comportamentos ou ações que um objeto pode realizar. No exemplo, apresentar ( ) é um método que exibe informações sobre a pessoa.
-

## Encapsulamento: Modificadores de acesso e métodos getter/setter

**Encapsulamento** é o princípio que permite ocultar os detalhes internos de uma classe, expondo apenas o necessário. Isso aumenta a segurança e a modularidade do código.

### Modificadores de acesso

- **public**: O membro é acessível de qualquer lugar.
- **private**: O membro é acessível apenas dentro da própria classe.
- **protected**: O membro é acessível dentro do mesmo pacote ou subclasses.
- **Sem modificador (default)**: Acessível apenas dentro do mesmo pacote.

### Exemplo de encapsulamento com getters e setters:

```
public class ContaBancaria {
    // Atributos privados
    private String titular;
    private double saldo;

    // Construtor
    public ContaBancaria(String titular) {
        this.titular = titular;
        this.saldo = 0.0;
    }

    // Método getter para titular
    public String getTitular() {
        return titular;
    }

    // Método setter para titular
    public void setTitular(String titular) {
        this.titular = titular;
    }

    // Método getter para saldo
    public double getSaldo() {
        return saldo;
    }

    // Métodos para manipular o saldo
    public void depositar(double valor) {
        if (valor > 0) {
            saldo += valor;
            System.out.println("Depósito de R$" + valor + " realizado com sucesso.");
        }
    }
}
```

```

        } else {
            System.out.println("Valor inválido para depósito.");
        }
    }

    public void sacar(double valor) {
        if (valor > 0 && saldo >= valor) {
            saldo -= valor;
            System.out.println("Saque de R$" + valor + " realizado com
sucesso.");
        } else {
            System.out.println("Saldo insuficiente ou valor inválido.");
        }
    }
}

```

#### Uso da classe ContaBancaria:

```

public class Banco {
    public static void main(String[] args) {
        ContaBancaria conta = new ContaBancaria("João Silva");

        conta.depositar(500);
        conta.sacar(200);

        System.out.println("Titular: " + conta.getTitular());
        System.out.println("Saldo atual: R$" + conta.getSaldo());
    }
}

```

#### Saída:

```

Depósito de R$500.0 realizado com sucesso.
Saque de R$200.0 realizado com sucesso.
Titular: João Silva
Saldo atual: R$300.0

```

#### Benefícios do encapsulamento:

- **Proteção dos dados:** Impede acesso direto aos atributos, evitando modificações indesejadas.

- **Flexibilidade:** Permite alterar a implementação interna sem afetar outras partes do código.
  - **Manutenção:** Facilita a manutenção e evolução do código.
- 

## Herança e polimorfismo

### Herança

A herança permite que uma classe (subclasse) herde atributos e métodos de outra classe (superclasse). Isso promove a reutilização de código e a criação de hierarquias de classes.

#### Exemplo de herança:

```
// Superclasse
public class Veiculo {
    protected String marca;
    protected String modelo;

    public void acelerar() {
        System.out.println("O veículo está acelerando.");
    }
}

// Subclasse
public class Carro extends Veiculo {
    private int numPortas;

    public Carro(String marca, String modelo, int numPortas) {
        this.marca = marca;
        this.modelo = modelo;
        this.numPortas = numPortas;
    }

    public void exibirDetalhes() {
        System.out.println("Marca: " + marca);
        System.out.println("Modelo: " + modelo);
        System.out.println("Número de portas: " + numPortas);
    }
}
```

#### Uso da classe Carro:

```
public class Concessionaria {  
    public static void main(String[] args) {  
        Carro carro = new Carro("Toyota", "Corolla", 4);  
        carro.acelerar(); // Método herdado de Veiculo  
        carro.exibirDetalhes();  
    }  
}
```

**Saída:**

```
O veículo está acelerando.  
Marca: Toyota  
Modelo: Corolla  
Número de portas: 4
```

## Polimorfismo

O polimorfismo permite que um objeto seja referenciado de múltiplas formas, geralmente através de uma superclasse ou interface. Isso significa que uma classe pode ser usada como se fosse sua própria classe ou como qualquer uma de suas superclasses.

### Sobrescrita de métodos (Override)

Permite que uma subclasse forneça uma implementação específica de um método que já existe na superclasse.

**Exemplo:**

```
public class Veiculo {  
    public void acelerar() {  
        System.out.println("O veículo está acelerando.");  
    }  
}  
  
public class Moto extends Veiculo {  
    @Override  
    public void acelerar() {  
        System.out.println("A moto está acelerando rapidamente.");  
    }  
}
```

**Uso do polimorfismo:**

```

public class TestePolimorfismo {
    public static void main(String[] args) {
        Veiculo veiculo1 = new Veiculo();
        Veiculo veiculo2 = new Moto(); // Polimorfismo

        veiculo1.acelerar(); // Chama o método da classe Veiculo
        veiculo2.acelerar(); // Chama o método sobrescrito na classe
        Moto
    }
}

```

**Saída:**

```

O veículo está acelerando.
A moto está acelerando rapidamente.

```

### **Sobrecarga de métodos (Overload)**

Permite que uma classe tenha vários métodos com o mesmo nome, mas com assinaturas diferentes (tipos e/ou número de parâmetros).

**Exemplo:**

```

public class Calculadora {
    public int somar(int a, int b) {
        return a + b;
    }

    public double somar(double a, double b) {
        return a + b;
    }

    public int somar(int a, int b, int c) {
        return a + b + c;
    }
}

```

**Uso da classe Calculadora:**



```
public class TesteCalculadora {
    public static void main(String[] args) {
        Calculadora calc = new Calculadora();

        System.out.println("Soma de 2 inteiros: " + calc.somar(5, 3));
        System.out.println("Soma de 2 doubles: " + calc.somar(5.5,
3.3));
        System.out.println("Soma de 3 inteiros: " + calc.somar(1, 2,
3));
    }
}
```

**Saída:**

```
Soma de 2 inteiros: 8
Soma de 2 doubles: 8.8
Soma de 3 inteiros: 6
```

---

## Interfaces e classes abstratas

### Interfaces

Uma interface em Java é um contrato que define um conjunto de métodos abstratos (sem implementação) que uma classe deve implementar. Uma classe pode implementar múltiplas interfaces.

**Exemplo de interface:**

```
public interface Animal {
    void emitirSom();
    void mover();
}
```

**Implementando a interface:**

```
public class Cachorro implements Animal {
    @Override
    public void emitirSom() {
        System.out.println("O cachorro late: Au au!");
    }
}
```

```

    }

    @Override
    public void mover() {
        System.out.println("O cachorro está correndo.");
    }
}

```

### Uso da classe Cachorro:

```

public class Zoologico {
    public static void main(String[] args) {
        Animal animal = new Cachorro();
        animal.emitirSom();
        animal.mover();
    }
}

```

### Saída:

```

O cachorro late: Au au!
O cachorro está correndo.

```

### Classes abstratas

Uma classe abstrata é uma classe que não pode ser instanciada diretamente e pode conter métodos abstratos (sem implementação) e concretos (com implementação). Serve como base para outras classes que precisam implementar os métodos abstratos.

### Exemplo de classe abstrata:

```

public abstract class FormaGeometrica {
    public abstract double calcularArea();
    public void exibirTipo() {
        System.out.println("Esta é uma forma geométrica.");
    }
}

```

### Subclasses que estendem a classe abstrata:

```

public class Circulo extends FormaGeometrica {
    private double raio;

    public Circulo(double raio) {
        this.raio = raio;
    }

    @Override
    public double calcularArea() {
        return Math.PI * raio * raio;
    }
}

public class Retangulo extends FormaGeometrica {
    private double largura;
    private double altura;

    public Retangulo(double largura, double altura) {
        this.largura = largura;
        this.altura = altura;
    }

    @Override
    public double calcularArea() {
        return largura * altura;
    }
}

```

### Uso das classes:

```

public class Geometria {
    public static void main(String[] args) {
        FormaGeometrica circulo = new Circulo(5);
        FormaGeometrica retangulo = new Retangulo(4, 6);

        circulo.exibirTipo();
        System.out.println("Área do círculo: " +
            circulo.calcularArea());

        retangulo.exibirTipo();
        System.out.println("Área do retângulo: " +
            retangulo.calcularArea());
    }
}

```

```
}
```

**Saída:**

```
Esta é uma forma geométrica.  
Área do círculo: 78.53981633974483  
Esta é uma forma geométrica.  
Área do retângulo: 24.0
```

### Diferenças entre interfaces e classes abstratas:

- **Herança múltipla:** Uma classe pode implementar múltiplas interfaces, mas só pode estender uma única classe (abstrata ou não).
- **Implementação de métodos:** Interfaces (antes do Java 8) não possuem implementação de métodos, enquanto classes abstratas podem ter métodos concretos.
- **Quando usar:**
  - **Interface:** Quando você quer definir um contrato que várias classes, não necessariamente relacionadas, podem implementar.
  - **Classe abstrata:** Quando você quer fornecer uma base comum com implementação parcial para classes relacionadas.

## Capítulo 7: Tratamento de exceções

O tratamento de exceções é um aspecto importante na programação em Java, permitindo que seu programa lide com situações inesperadas ou erros de forma controlada. Compreender como funcionam as exceções e como manipulá-las adequadamente é essencial para escrever código robusto e resiliente.

---

### Introdução às exceções

#### O que são exceções?

Em Java, uma exceção é um evento que interrompe o fluxo normal de execução de um programa. Elas ocorrem quando uma situação anormal ou inesperada é detectada durante a execução, como tentar acessar um índice inválido de um array, dividir por zero ou trabalhar com objetos nulos.

#### Exemplo de exceção:

```
public class ExemploExcecao {  
    public static void main(String[] args) {  
        int[] numeros = {1, 2, 3};  
        System.out.println(numeros[5]); // Tentativa de acessar índice  
        inválido  
    }  
}
```

Saída:

```
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: 5  
at ExemploExcecao.main(ExemploExcecao.java:4)
```

### Diferença entre erros e exceções

- **Erros (Errors):** Representam problemas sérios que uma aplicação razoável não deve tentar capturar. São condições anormais que geralmente indicam problemas no ambiente de execução, como falta de memória ou falhas de sistema. Em Java, são subclasses da classe `Error`.
- **Exceções (Exceptions):** Indicam problemas que podem ser previstos e tratados pelo programa. São condições que um programa pode querer capturar e manipular, como entrada de dados inválida ou falta de um arquivo esperado. São subclasses da classe `Exception`.

### Hierarquia de classes de exceção

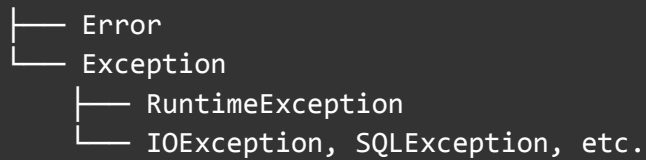
A classe `Throwable` é a superclasse de todas as classes que podem ser lançadas como exceções ou erros. A hierarquia básica é a seguinte:

#### `Throwable`

- **Error**
  - Problemas sérios (geralmente não recuperáveis)
- **Exception**
  - **RuntimeException**
    - Exceções não verificadas (unchecked exceptions)
  - Outras exceções
    - Exceções verificadas (checked exceptions)

Diagrama simplificado:

```
Throwable
```



## Exceções verificadas vs. não verificadas

### Exceções verificadas (Checked Exceptions):

- O compilador verifica se elas são tratadas ou declaradas.
- Devem ser capturadas com try-catch ou declaradas com throws na assinatura do método.
- Exemplo: IOException, SQLException.

### Exceções não verificadas (Unchecked Exceptions):

- O compilador não exige que sejam tratadas ou declaradas.
- Geralmente resultam de erros de programação.
- Subclasses de RuntimeException.
- Exemplo: NullPointerException, ArrayIndexOutOfBoundsException.

---

## Blocos try, catch, finally: Como capturar e tratar exceções

O Java fornece estruturas específicas para lidar com exceções e permitir que o programa continue executando de forma controlada.

### Estrutura básica

```
try {  
    // Código que pode lançar uma exceção  
} catch (TipoDeExcecao e) {  
    // Tratamento da exceção  
} finally {  
    // Bloco opcional que sempre é executado  
}
```

### Exemplo prático:

```
import java.util.Scanner;  
  
public class DivisaoSegura {  
    public static void main(String[] args) {  
        Scanner scanner = new Scanner(System.in);
```

```

    try {
        System.out.print("Digite o numerador: ");
        int numerador = scanner.nextInt();

        System.out.print("Digite o denominador: ");
        int denominador = scanner.nextInt();

        int resultado = numerador / denominador;
        System.out.println("Resultado: " + resultado);
    } catch (ArithmeticException e) {
        System.out.println("Erro: Divisão por zero não é permitida.");
    } catch (Exception e) {
        System.out.println("Erro inesperado: " + e.getMessage());
    } finally {
        scanner.close();
        System.out.println("Recursos liberados.");
    }
}

```

#### Possíveis saídas:

- **Caso normal:**

```

Digite o numerador: 10
Digite o denominador: 2
Resultado: 5
Recursos liberados.

```

- **Divisão por zero:**

```

Digite o numerador: 10
Digite o denominador: 0
Erro: Divisão por zero não é permitida.
Recursos liberados.

```

#### Explicação dos blocos

- **try:** Contém o código que pode lançar uma exceção.

- **catch:** Captura e trata a exceção lançada no bloco `try`. Pode haver múltiplos blocos `catch` para diferentes tipos de exceções.
- **finally:** Bloco opcional que é executado após o `try` e os `catch`, independentemente de uma exceção ter sido lançada ou não. Geralmente usado para liberar recursos.

### Capturando múltiplas exceções

Você pode capturar várias exceções em blocos separados ou combinar exceções relacionadas.

#### Exemplo:

```
try {  
    // Código que pode lançar várias exceções  
} catch (IOException e) {  
    // Tratamento para IOException  
} catch (SQLException e) {  
    // Tratamento para SQLException  
} catch (Exception e) {  
    // Tratamento genérico para outras exceções  
}
```

Desde o **Java 7**, é possível capturar múltiplas exceções no mesmo bloco `catch` usando o operador `|`.

#### Exemplo:

```
try {  
    // Código que pode lançar várias exceções  
} catch (IOException | SQLException e) {  
    // Tratamento para IOException ou SQLException  
}
```

### A ordem dos blocos catch importa

Os blocos `catch` devem ser ordenados do mais específico para o mais genérico. Caso contrário, o compilador gerará um erro, pois um bloco genérico pode capturar exceções que deveriam ser tratadas em blocos mais específicos.

#### Exemplo incorreto:



```
try {  
    // Código  
} catch (Exception e) {  
    // Tratamento genérico  
} catch (IOException e) {  
    // Erro de compilação: este bloco nunca será alcançado  
}
```

---

## Lançando exceções: Uso da palavra-chave throw e criação de exceções personalizadas

### Lançando exceções com throw

A palavra-chave throw é usada para lançar explicitamente uma exceção em um método ou bloco de código.

#### Exemplo:

```
public class VerificadorIdade {  
    public void verificar(int idade) {  
        if (idade < 18) {  
            throw new IllegalArgumentException("Idade deve ser maior ou  
igual a 18.");  
        } else {  
            System.out.println("Idade válida.");  
        }  
    }  
}
```

#### Uso da classe:

```
public class TesteVerificador {  
    public static void main(String[] args) {  
        VerificadorIdade verificador = new VerificadorIdade();  
        try {  
            verificador.verificar(16);  
        } catch (IllegalArgumentException e) {  
            System.out.println("Exceção capturada: " + e.getMessage());  
        }  
    }  
}
```

```
}
```

**Saída:**

```
Exceção capturada: Idade deve ser maior ou igual a 18.
```

### Declarando exceções com throws

Se um método pode lançar uma exceção verificada e não a captura internamente, ele deve declará-la usando throws em sua assinatura.

**Exemplo:**

```
public void metodo() throws IOException {  
    // Código que pode lançar IOException  
}
```

### Criando exceções personalizadas

Você pode criar suas próprias classes de exceção para representar situações específicas em seu programa. As exceções personalizadas devem estender a classe Exception (para exceções verificadas) ou RuntimeException (para exceções não verificadas).

**Exemplo de exceção personalizada:**

```
public class SaldoInsuficienteException extends Exception {  
    public SaldoInsuficienteException(String mensagem) {  
        super(mensagem);  
    }  
}
```

**Uso da exceção personalizada:**

```
public class Conta {  
    private double saldo;  
  
    public void sacar(double valor) throws SaldoInsuficienteException {
```

```

        if (valor > saldo) {
            throw new SaldoInsuficienteException("Saldo insuficiente
para saque de R$" + valor);
        } else {
            saldo -= valor;
            System.out.println("Saque de R$" + valor + " realizado com
sucesso.");
        }
    }

    // Métodos adicionais (depositar, consultar saldo, etc.)
}

```

**Classe principal:**

```

public class Banco {
    public static void main(String[] args) {
        Conta conta = new Conta();
        // Suponha que o saldo inicial seja zero

        try {
            conta.sacar(100);
        } catch (SaldoInsuficienteException e) {
            System.out.println("Erro: " + e.getMessage());
        }
    }
}

```

**Saída:**

```

Erro: Saldo insuficiente para saque de R$100.0

```

**Exceções verificadas vs. não verificadas em exceções personalizadas**

- Para criar uma exceção verificada, estenda a classe `Exception`.
- Para criar uma exceção não verificada, estenda a classe `RuntimeException`.

**Exemplo de exceção não verificada:**

```
public class MinhaRuntimeException extends RuntimeException {
    public MinhaRuntimeException(String mensagem) {
        super(mensagem);
    }
}
```

---

## Boas práticas no tratamento de erros

### 1. Capture apenas exceções que você pode tratar

Não capture exceções apenas para suprimir erros. Se você não puder tratar a exceção de maneira significativa, deixe-a propagar.

**Evite:**

```
try {
    // Código
} catch (Exception e) {
    // Não faz nada ou imprime mensagem genérica
}
```

### 2. Não sufoque exceções

Não ignore exceções sem pelo menos registrar o erro ou tomar alguma ação.

**Evite:**

```
try {
    // Código
} catch (IOException e) {
    // Nada aqui
}
```

**Prefira:**

```
try {
    // Código
} catch (IOException e) {
    e.printStackTrace(); // Ou registre o erro adequadamente
}
```

```
}
```

### 3. Especifique exceções de maneira precisa

Capture exceções específicas em vez de usar `Exception` genérico.

**Evite:**

```
try {  
    // Código  
} catch (Exception e) {  
    // Tratamento  
}
```

**Prefira:**

```
try {  
    // Código  
} catch (IOException e) {  
    // Tratamento para IOException  
} catch (SQLException e) {  
    // Tratamento para SQLException  
}
```

### 4. Limpe recursos no bloco `finally` ou use `try-with-resources`

Certifique-se de liberar recursos como streams, conexões de banco de dados, etc.

**Exemplo com `finally`:**

```
FileReader fr = null;  
try {  
    fr = new FileReader("arquivo.txt");  
    // Operações com o arquivo  
} catch (IOException e) {  
    e.printStackTrace();  
} finally {  
    if (fr != null) {  
        try {
```

```

        fr.close();
    } catch (IOException e) {
        e.printStackTrace();
    }
}
}

```

## 5. Utilize try-with-resources (Java 7+)

Simplifica o gerenciamento de recursos que implementam a interface `AutoCloseable`.

**Exemplo:**

```

try (FileReader fr = new FileReader("arquivo.txt")) {
    // Operações com o arquivo
} catch (IOException e) {
    e.printStackTrace();
}

```

## 6. Forneça mensagens de erro úteis

Ao lançar exceções, inclua mensagens claras que ajudem a diagnosticar o problema.

**Exemplo:**

```

throw new IllegalArgumentException("O parâmetro 'nome' não pode ser nulo.");

```

## 7. Não use exceções para controle de fluxo

Exceções devem ser usadas para situações excepcionais, não como parte da lógica normal do programa.

**Evite:**

```

try {
    int valor = Integer.parseInt("texto"); // Isso lançará uma exceção
} catch (NumberFormatException e) {
    // Use a exceção como lógica de fluxo
}

```

**Prefira validar os dados antes:**

```
String entrada = "texto";
if (entrada.matches("\\d+")) {
    int valor = Integer.parseInt(entrada);
} else {
    System.out.println("Entrada inválida: não é um número.");
}
```

## Capítulo 8: Entrada e Saída (I/O)

A entrada e saída (I/O) em Java é um aspecto fundamental para interagir com o usuário, ler e gravar dados em arquivos, e realizar comunicação em rede. Neste capítulo, exploraremos as operações básicas de I/O, como leitura e escrita no console, manipulação de arquivos utilizando classes específicas, compreensão do conceito de streams, e a serialização de objetos para armazenamento persistente.

---

### Operações básicas de I/O: Leitura e escrita de dados no console

A interação com o usuário através do console é comum em aplicações Java, especialmente em programas de linha de comando. Para realizar operações de leitura e escrita no console, o Java fornece classes e métodos dedicados.

#### Escrita no console usando `System.out`

O fluxo de saída padrão em Java é representado pelo objeto `System.out`, que é uma instância da classe `PrintStream`. Os métodos mais utilizados são `print()` e `println()`.

**Exemplo:**

```
public class EscritaConsole {
    public static void main(String[] args) {
        System.out.print("Este texto será impresso sem quebra de
linha.");
        System.out.println("Este texto será impresso com quebra de
linha.");
        System.out.println("Número inteiro: " + 42);
        System.out.printf("Formatação de número decimal: %.2f%n",
3.1415);
    }
}
```

```
}  
}
```

### Saída:

```
Este texto será impresso sem quebra de linha.Este texto será impresso  
com quebra de linha.  
Número inteiro: 42  
Formatação de número decimal: 3.14
```

### Observações:

- `print()`: Imprime o texto sem adicionar uma nova linha no final.
- `println()`: Imprime o texto e adiciona uma nova linha no final.
- `printf()`: Permite formatar a saída utilizando especificadores de formato, similar à linguagem C.

### Leitura de dados do console usando Scanner

A classe `Scanner`, presente no pacote `java.util`, é amplamente utilizada para ler dados do console de forma simples e eficiente.

### Exemplo:

```
import java.util.Scanner;  
  
public class LeituraScanner {  
    public static void main(String[] args) {  
        Scanner scanner = new Scanner(System.in);  
  
        System.out.print("Digite seu nome: ");  
        String nome = scanner.nextLine();  
  
        System.out.print("Digite sua idade: ");  
        int idade = scanner.nextInt();  
  
        System.out.print("Digite sua altura (em metros): ");  
        double altura = scanner.nextDouble();  
  
        System.out.println("\n--- Dados Informados ---");  
        System.out.println("Nome: " + nome);  
        System.out.println("Idade: " + idade + " anos");  
    }  
}
```



```
        System.out.println("Altura: " + altura + " m");

        scanner.close();
    }
}
```

### Exemplo de entrada:

```
Digite seu nome: Ana
Digite sua idade: 28
Digite sua altura (em metros): 1.65
```

### Saída:

```
--- Dados Informados ---
Nome: Ana
Idade: 28 anos
Altura: 1.65 m
```

### Métodos comuns da classe Scanner:

- `nextLine()`: Lê uma linha inteira de texto.
- `next()`: Lê a próxima sequência de caracteres até um espaço ou quebra de linha.
- `nextInt()`: Lê um valor inteiro.
- `nextDouble()`: Lê um valor decimal (double).
- `hasNext()`: Verifica se há mais tokens (entradas) disponíveis.

### Observações:

- Sempre feche o objeto Scanner após o uso com `scanner.close()` para liberar recursos.
- Em algumas situações, ao misturar `nextLine()` com outros métodos como `nextInt()`, pode ser necessário consumir a quebra de linha pendente usando `scanner.nextLine()` adicional.

### Leitura de dados do console usando `BufferedReader`

Antes do Java 5, a classe `BufferedReader` era comumente utilizada para leitura de dados do console. Embora `Scanner` seja mais simples, é útil conhecer `BufferedReader` para casos específicos.

### Exemplo:

```
import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;

public class LeituraBufferedReader {
    public static void main(String[] args) {
        BufferedReader reader = new BufferedReader(new
        InputStreamReader(System.in));

        try {
            System.out.print("Digite seu nome: ");
            String nome = reader.readLine();

            System.out.print("Digite sua idade: ");
            int idade = Integer.parseInt(reader.readLine());

            System.out.println("\nNome: " + nome);
            System.out.println("Idade: " + idade);
        } catch (IOException e) {
            System.out.println("Erro na leitura dos dados: " +
            e.getMessage());
        }
    }
}
```

### Observações:

- InputStreamReader converte o fluxo de bytes (System.in) em caracteres.
- BufferedReader lê texto de um fluxo de entrada de caracteres, armazenando em buffer para fornecer uma leitura eficiente.
- É necessário tratar a exceção IOException.

---

### Trabalhando com arquivos

Manipular arquivos é uma tarefa comum em muitas aplicações, seja para ler dados de configurações, salvar registros ou processar grandes volumes de informação.

#### Classe File

A classe `File`, do pacote `java.io`, representa um arquivo ou diretório no sistema de arquivos. Ela fornece métodos para inspecionar propriedades de arquivos e diretórios, bem como para criar, renomear e excluir arquivos.

#### Exemplo de uso da classe `File`:

```
import java.io.File;
import java.io.IOException;

public class ExemploFile {
    public static void main(String[] args) {
        File arquivo = new File("exemplo.txt");

        try {
            if (arquivo.createNewFile()) {
                System.out.println("Arquivo criado: " +
arquivo.getName());
            } else {
                System.out.println("Arquivo já existe.");
            }

            System.out.println("Caminho absoluto: " +
arquivo.getAbsolutePath());
            System.out.println("É legível: " + arquivo.canRead());
            System.out.println("É gravável: " + arquivo.canWrite());
            System.out.println("Tamanho: " + arquivo.length() + "
bytes");

            // Renomear arquivo
            File novoArquivo = new File("novo_exemplo.txt");
            if (arquivo.renameTo(novoArquivo)) {
                System.out.println("Arquivo renomeado para: " +
novoArquivo.getName());
            } else {
                System.out.println("Falha ao renomear o arquivo.");
            }

            // Excluir arquivo
            if (novoArquivo.delete()) {
                System.out.println("Arquivo excluído.");
            } else {
                System.out.println("Falha ao excluir o arquivo.");
            }
        } catch (IOException e) {
            System.out.println("Ocorreu um erro: " + e.getMessage());
        }
    }
}
```

```
}  
}
```

### Métodos comuns da classe File:

- `createNewFile()`: Cria um novo arquivo vazio se não existir.
- `exists()`: Verifica se o arquivo ou diretório existe.
- `isFile()`: Verifica se é um arquivo.
- `isDirectory()`: Verifica se é um diretório.
- `listFiles()`: Retorna um array de File com os arquivos/diretórios contidos.
- `delete()`: Exclui o arquivo ou diretório.
- `renameTo(File dest)`: Renomeia o arquivo para o destino especificado.

### Leitura de arquivos usando FileReader e BufferedReader

Para ler o conteúdo de um arquivo texto, combinamos FileReader com BufferedReader.

#### Exemplo:

```
import java.io.BufferedReader;  
import java.io.FileReader;  
import java.io.IOException;  
  
public class LeituraArquivo {  
    public static void main(String[] args) {  
        String nomeArquivo = "dados.txt";  
  
        try (BufferedReader br = new BufferedReader(new  
FileReader(nomeArquivo))) {  
            String linha;  
            while ((linha = br.readLine()) != null) {  
                System.out.println(linha);  
            }  
        } catch (IOException e) {  
            System.out.println("Erro ao ler o arquivo: " +  
e.getMessage());  
        }  
    }  
}
```

#### Explicação:

- FileReader lê fluxos de caracteres de um arquivo.
- BufferedReader lê texto de forma eficiente, permitindo ler linhas inteiras com `readLine()`.
- O bloco `try-with-resources` garante que os recursos serão fechados automaticamente.

## Escrita em arquivos usando `FileWriter` e `BufferedWriter`

Para escrever dados em um arquivo, utilizamos `FileWriter` e `BufferedWriter`.

### Exemplo:

```
import java.io.BufferedWriter;
import java.io.FileWriter;
import java.io.IOException;

public class EscritaArquivo {
    public static void main(String[] args) {
        String nomeArquivo = "saida.txt";

        try (BufferedWriter bw = new BufferedWriter(new
FileWriter(nomeArquivo, true))) {
            bw.write("Linha 1: Exemplo de escrita em arquivo.");
            bw.newLine();
            bw.write("Linha 2: Continuação do exemplo.");
            bw.newLine();
            System.out.println("Dados gravados com sucesso.");
        } catch (IOException e) {
            System.out.println("Erro ao escrever no arquivo: " +
e.getMessage());
        }
    }
}
```

### Observações:

- O segundo parâmetro em `FileWriter(nomeArquivo, true)` indica se devemos **anexar** (`true`) ou sobrescrever (`false`) o arquivo.
  - `BufferedWriter` oferece eficiência na escrita, armazenando os dados em buffer antes de gravá-los no disco.
  - Utilize `newLine()` para adicionar quebras de linha de forma independente do sistema operacional.
-

## Streams em Java

### Conceito de fluxos de entrada e saída

Em Java, um **stream** (fluxo) é uma sequência de dados que pode ser lida (entrada) ou escrita (saída). Os streams abstraem as operações de I/O, permitindo que você trabalhe com diferentes fontes e destinos de dados (arquivos, rede, memória) de maneira consistente.

### Classes `InputStream` e `OutputStream`

- **`InputStream`**: Classe abstrata que representa um fluxo de entrada de bytes. Fornece métodos para ler bytes de uma fonte de dados.
- **`OutputStream`**: Classe abstrata que representa um fluxo de saída de bytes. Fornece métodos para escrever bytes em um destino de dados.

### Hierarquia de classes de streams

As classes `InputStream` e `OutputStream` possuem diversas subclasses para diferentes tipos de fontes e destinos.

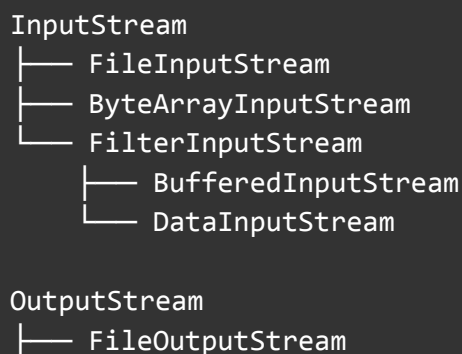
#### Principais subclasses de `InputStream`:

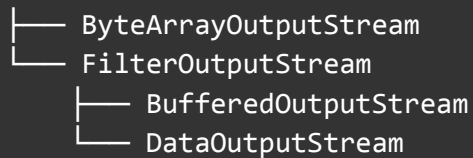
- `FileInputStream`: Lê bytes de um arquivo.
- `ByteArrayInputStream`: Lê bytes de um array de bytes.
- `FilterInputStream`: Classe base para streams que filtram dados (e.g., `BufferedInputStream`, `DataInputStream`).

#### Principais subclasses de `OutputStream`:

- `FileOutputStream`: Escreve bytes em um arquivo.
- `ByteArrayOutputStream`: Escreve bytes em um array de bytes.
- `FilterOutputStream`: Classe base para streams que filtram dados (e.g., `BufferedOutputStream`, `DataOutputStream`).

#### Diagrama simplificado:





## Uso de FileInputStream e FileOutputStream

Essas classes permitem ler e escrever bytes diretamente em arquivos.

### Exemplo de leitura de bytes:

```
import java.io.FileInputStream;
import java.io.IOException;

public class LeituraBytes {
    public static void main(String[] args) {
        String nomeArquivo = "imagem.jpg";

        try (FileInputStream fis = new FileInputStream(nomeArquivo)) {
            int byteLido;
            while ((byteLido = fis.read()) != -1) {
                // Processar o byte lido (por exemplo, armazenar em um
                array)
            }
            System.out.println("Leitura concluída.");
        } catch (IOException e) {
            System.out.println("Erro ao ler o arquivo: " +
            e.getMessage());
        }
    }
}
```

### Exemplo de escrita de bytes:

```
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;

public class CopiaArquivoBinario {
    public static void main(String[] args) {
        String origem = "imagem.jpg";
        String destino = "copia_imagem.jpg";
```

```

try (FileInputStream fis = new FileInputStream(origem);
    FileOutputStream fos = new FileOutputStream(destino)) {

    byte[] buffer = new byte[1024];
    int bytesLidos;
    while ((bytesLidos = fis.read(buffer)) != -1) {
        fos.write(buffer, 0, bytesLidos);
    }
    System.out.println("Cópia concluída.");
} catch (IOException e) {
    System.out.println("Erro ao copiar o arquivo: " +
e.getMessage());
}
}
}

```

#### Observações:

- Ao trabalhar com arquivos binários (imagens, vídeos, etc.), é necessário ler e escrever bytes, não caracteres.
- Utilizar um buffer (como um array de bytes) aumenta a eficiência ao reduzir o número de operações de I/O.

#### Streams de caracteres vs. streams de bytes

- **Streams de caracteres** (Reader e Writer): Projetados para trabalhar com dados de texto (caracteres Unicode). Exemplos: FileReader, BufferedReader, FileWriter, BufferedWriter.
- **Streams de bytes** (InputStream e OutputStream): Projetados para trabalhar com dados binários (bytes). Exemplos: FileInputStream, FileOutputStream.

### Serialização de objetos

#### Conceito de serialização

Serialização é o processo de converter um objeto em uma sequência de bytes, permitindo que ele seja armazenado em um arquivo ou transmitido pela rede. A desserialização é o processo inverso, reconstruindo o objeto original a partir dos bytes.

#### Implementação da interface `Serializable`

Para que um objeto seja serializável, sua classe deve implementar a interface `Serializable`, que é uma interface de marcação (não possui métodos).



### Exemplo de classe serializável:

```
import java.io.Serializable;

public class Pessoa implements Serializable {
    private static final long serialVersionUID = 1L;

    private String nome;
    private int idade;

    public Pessoa(String nome, int idade) {
        this.nome = nome;
        this.idade = idade;
    }

    // Getters e setters
    public String getNome() {
        return nome;
    }

    public int getIdade() {
        return idade;
    }
}
```

### Observações:

- É uma boa prática definir um serialVersionUID para a classe, que é um identificador único da versão da classe.

### Uso de ObjectOutputStream e ObjectInputStream

Essas classes permitem gravar e ler objetos serializáveis em arquivos ou outros streams.

### Exemplo de salvando (serializando) objetos em um arquivo:

```
import java.io.FileOutputStream;
import java.io.IOException;
import java.io.ObjectOutputStream;

public class SerializacaoObjeto {
    public static void main(String[] args) {
        Pessoa pessoa = new Pessoa("Carlos", 35);
```

```

        try (FileOutputStream fos = new FileOutputStream("pessoa.ser");
            ObjectOutputStream oos = new ObjectOutputStream(fos)) {

            oos.writeObject(pessoa);
            System.out.println("Objeto serializado com sucesso.");
        } catch (IOException e) {
            System.out.println("Erro ao serializar o objeto: " +
e.getMessage());
        }
    }
}

```

**Exemplo de carregando (desserializando) objetos de um arquivo:**

```

import java.io.FileInputStream;
import java.io.IOException;
import java.io.ObjectInputStream;

public class DesserializacaoObjeto {
    public static void main(String[] args) {
        try (FileInputStream fis = new FileInputStream("pessoa.ser");
            ObjectInputStream ois = new ObjectInputStream(fis)) {

            Pessoa pessoa = (Pessoa) ois.readObject();
            System.out.println("Objeto desserializado:");
            System.out.println("Nome: " + pessoa.getNome());
            System.out.println("Idade: " + pessoa.getIdade());
        } catch (IOException | ClassNotFoundException e) {
            System.out.println("Erro ao desserializar o objeto: " +
e.getMessage());
        }
    }
}

```

**Saída:**

```

Objeto desserializado:
Nome: Carlos
Idade: 35

```

**Considerações importantes sobre serialização:**

- **Compatibilidade de versão:** Se a classe mudar após a serialização (adicionar ou remover atributos), pode ocorrer `InvalidClassException` durante a desserialização.
- **Atributos transient:** Atributos marcados como `transient` não são serializados.

**Exemplo:**

```
public class Usuario implements Serializable {  
    private static final long serialVersionUID = 1L;  
  
    private String username;  
    private transient String senha; // Não será serializada  
  
    // Construtor, getters e setters  
}
```

- **Segurança:** Serialização pode expor dados sensíveis. Tenha cuidado ao serializar objetos que contêm informações confidenciais.

## Conclusão

Você chegou ao final do **Guia Rápido de Java!**

Esta é uma conquista na sua jornada como desenvolvedor. Ao longo dos capítulos, navegamos pelos elementos fundamentais que formam a base da programação em Java, desde os conceitos mais básicos até tópicos mais avançados que são essenciais para o desenvolvimento de aplicações modernas.

Recapitulando, você aprendeu sobre:

- **Sintaxe e Estrutura Básica:** Estabeleceu uma base sólida para escrever e compreender códigos Java.
- **Tipos de Dados e Variáveis:** Entendeu como manipular informações e realizar conversões entre diferentes tipos.
- **Operadores e Expressões:** Dominou as ferramentas para realizar cálculos e operações lógicas.
- **Estruturas de Controle de Fluxo:** Aprendeu a direcionar o fluxo de execução dos programas de forma eficiente.
- **Arrays e Coleções:** Descobriu como gerenciar conjuntos de dados de maneira eficaz.
- **Programação Orientada a Objetos:** Absorveu os princípios que permitem criar códigos mais organizados e reutilizáveis.
- **Tratamento de Exceções:** Desenvolveu habilidades para tornar seus programas mais robustos e confiáveis.
- **Entrada e Saída (I/O):** Expandiu as capacidades de interação dos seus programas com o mundo externo.

Este guia foi projetado para não apenas ensinar os fundamentos, mas também inspirar confiança em suas habilidades para continuar explorando e aprendendo. A programação é uma disciplina em constante evolução, e o conhecimento adquirido aqui é fundamental para desafios mais complexos.

## Conclusão e próximos passos

A tecnologia é uma área dinâmica e repleta de oportunidades. Com dedicação e curiosidade, você pode não apenas acompanhar as mudanças, mas também ser parte ativa da inovação. Lembre-se de que cada programador experiente já foi um iniciante como você. O importante é continuar aprendendo, experimentando e nunca perder a paixão pelo conhecimento.

Obrigado por escolher este guia como seu companheiro de aprendizado. Desejo a você muito sucesso em sua carreira e que o Java seja uma ferramenta poderosa em suas realizações futuras.

E claro, se você quiser aprender Java do básico ao avançado, deixo aqui a indicação do nosso curso completo:

### [\*\*Curso de Java do Básico ao Avançado\*\*](#)

O curso tem quase 30 horas de duração, diversos exercícios e projetos para você dominar a linguagem.

#### Inclui:

- |                                  |                                    |
|----------------------------------|------------------------------------|
| ✓ <b>Materiais para download</b> | ✓ <b>Atualizações periódicas</b>   |
| ✓ <b>Aulas práticas</b>          | ✓ <b>Reconhecimento de mercado</b> |
| ✓ <b>321 aulas</b>               | ✓ <b>Exercícios</b>                |
| ✓ <b>27h 29min de curso</b>      | ✓ <b>Repositório no GitHub</b>     |
| ✓ <b>15 projetos completos</b>   | ✓ <b>Grupo no Discord</b>          |